

CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

Every Data Structure So Far

	Unsorted Array	Sorted Array	Unsorted List	Sorted List	Binary Tree	BST	AVL
Find	$O(n)$	$O(\lg(n))$	$O(n)$	$O(n)$	$O(n)$	$O(h)$	$O(\lg n)$
Insert	$O(1)$	find + $O(n)$	$O(1)$	find + $O(1)$	$O(1)$	find + $O(1)$	find + $O(1)$
Remove	find + $O(1)$	find + $O(n)$	find + $O(1)$	find + $O(1)$	find + $O(1)$	find + $O(1)$	find + $O(\lg n)$
Traverse	$O(n)$						

Range-based Searches

Q: Consider points in 1D: $\mathbf{p} = \{p_1, p_2, \dots, p_n\}$.
...what points fall in $[11, 42]$?

Tree construction:

- Balanced BST
- Leaf nodes are values
- Inner nodes are bounds

Range-based Searches

Balanced BSTs are useful structures for range-based and nearest-neighbor searches.

create sorted array
 $O(n \lg n)$

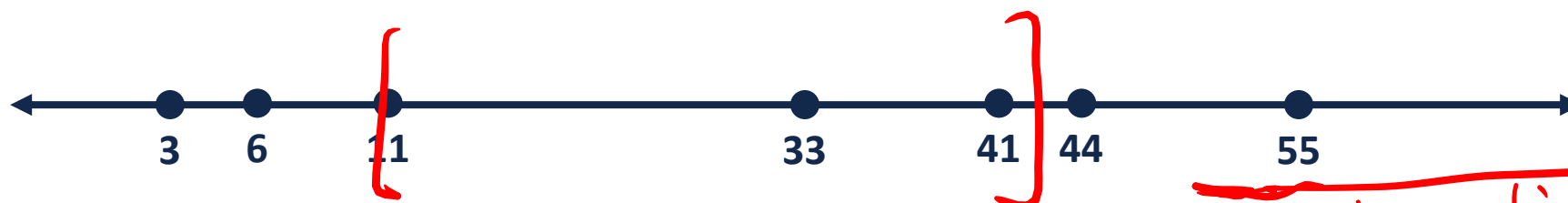
Q: Consider points in 1D: $p = \{p_1, p_2, \dots, p_n\}$.
...what points fall in $[11, 42]$?

find $O(\lg n)$

access k elements

$O(k)$

Ex:



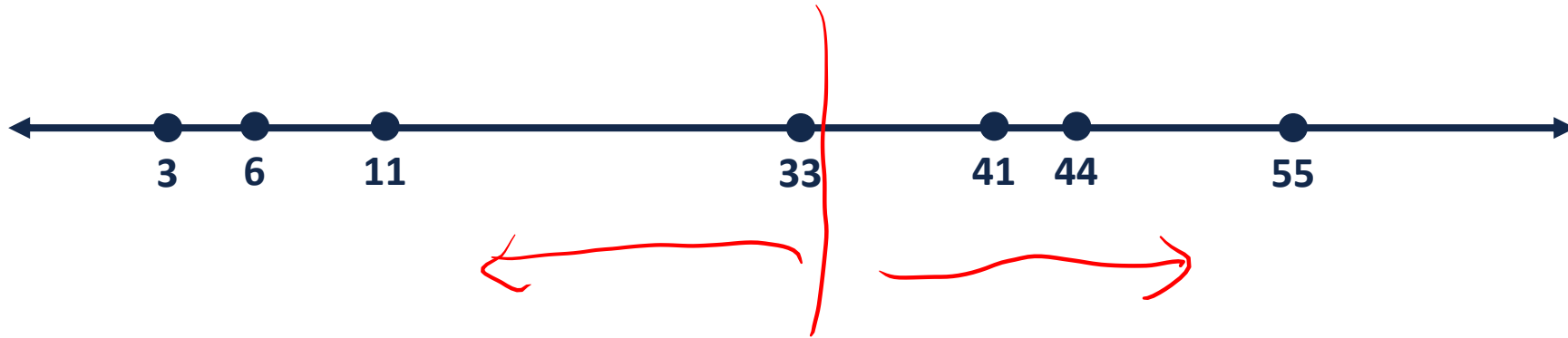
$(11, 33, 41)$

running time (array)
 $O(n \lg n + k)$

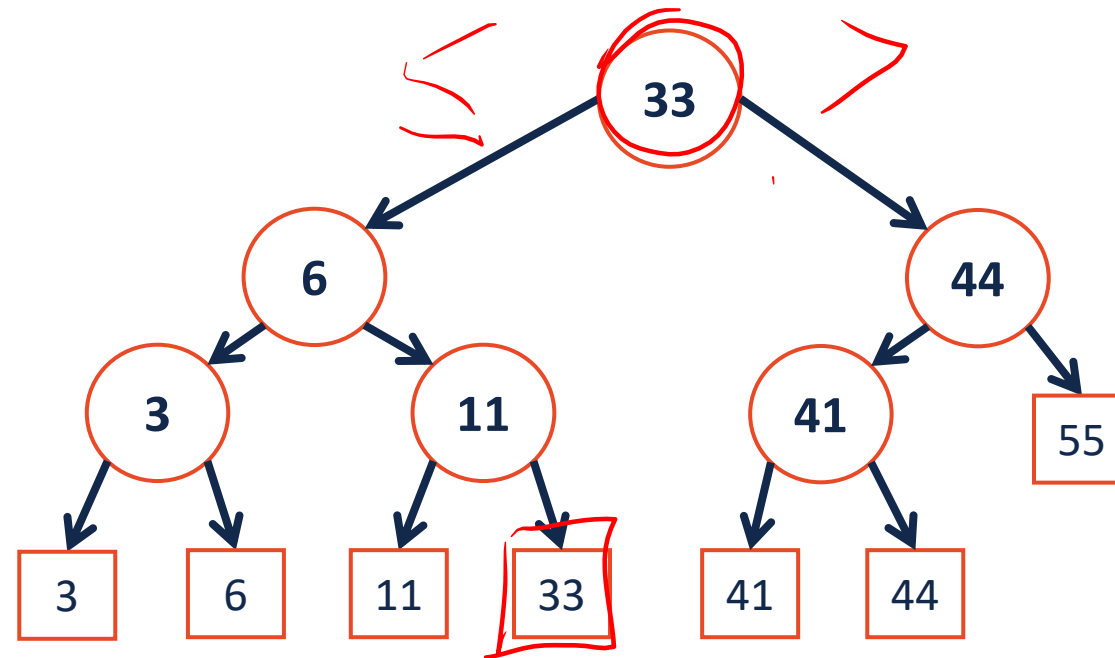
Range-based Searches

Q: Consider points in 1D: $\mathbf{p} = \{p_1, p_2, \dots, p_n\}$.
...what points fall in $[11, 42]$?

Ex:

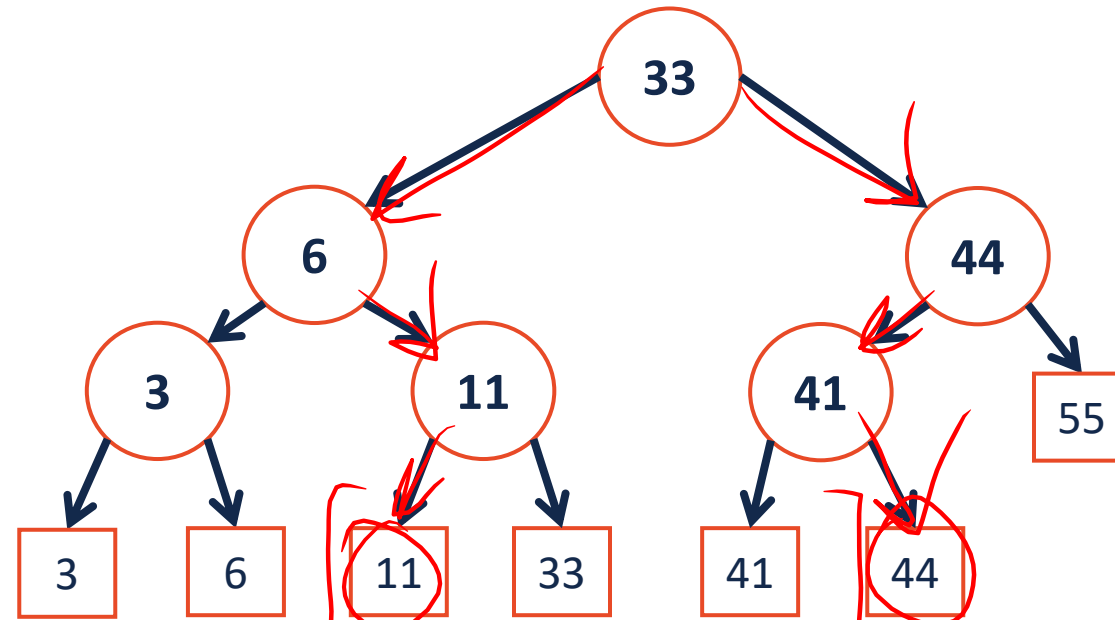


Range-based Searches



Range-based Searches

Q: Consider points in 1D: $\mathbf{p} = \{p_1, p_2, \dots, p_n\}$.
...what points fall in $[11, 42]$?



$P_{low} = 11$
 $P_{high} = 42$

in-order traversal

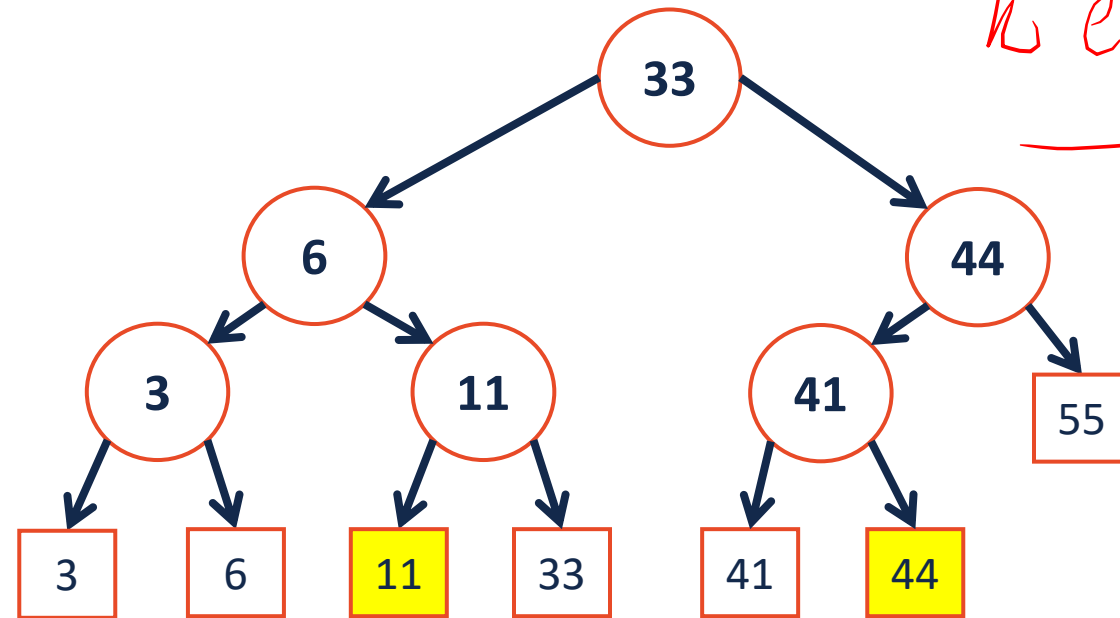
Running Time

insertion $O(n \lg n)$

find $2 * O(\lg n)$

k elements $O(k)$

$O(n \lg n + k)$



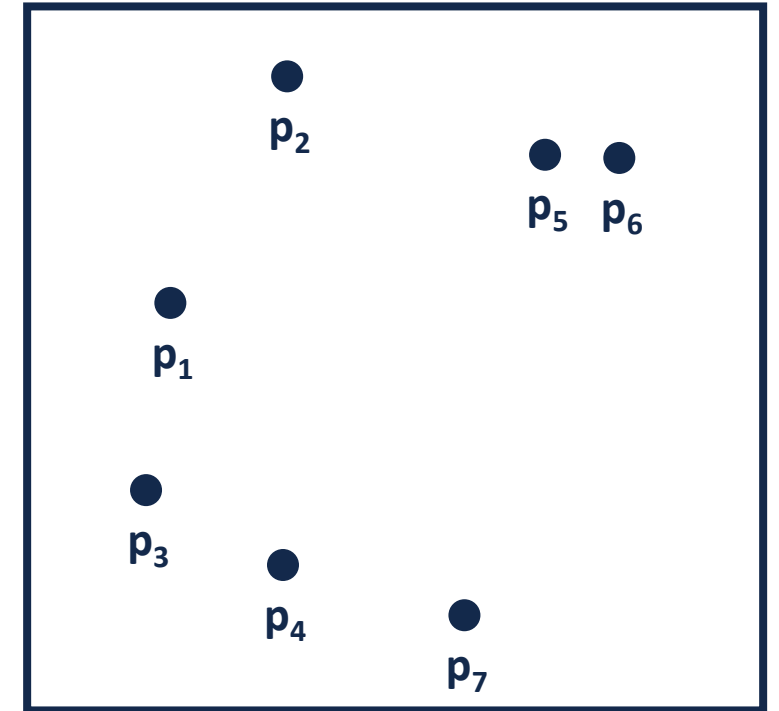
create link list

Range-based Searches

Consider points in 2D: $\mathbf{p} = \{p_1, p_2, \dots, p_n\}$.

Q: What points are in the rectangle:
[$(x_1, y_1), (x_2, y_2)$]?

Q: What is the nearest point to (x_1, y_1) ?



Range-based Searches

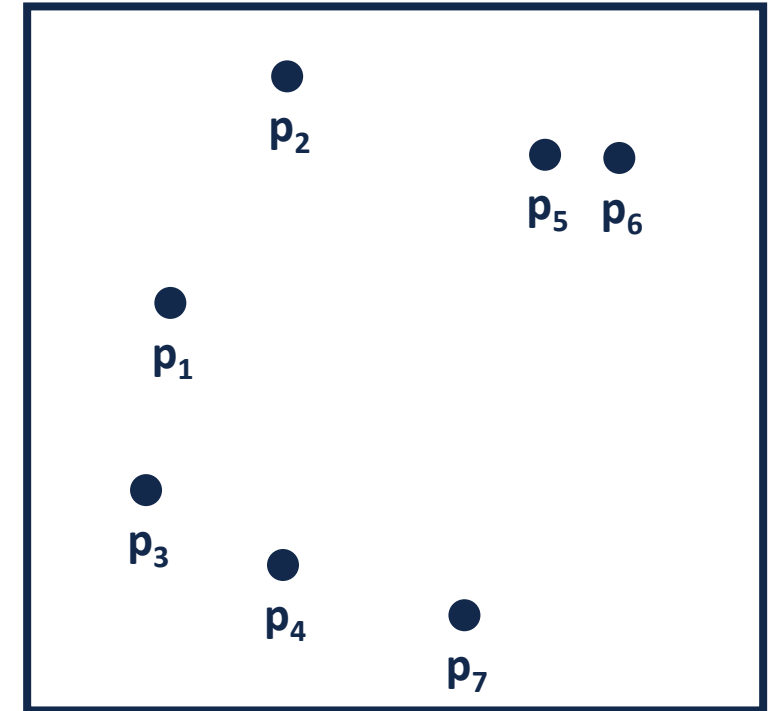
Consider points in 2D: $\mathbf{p} = \{p_1, p_2, \dots, p_n\}$.

Tree construction:

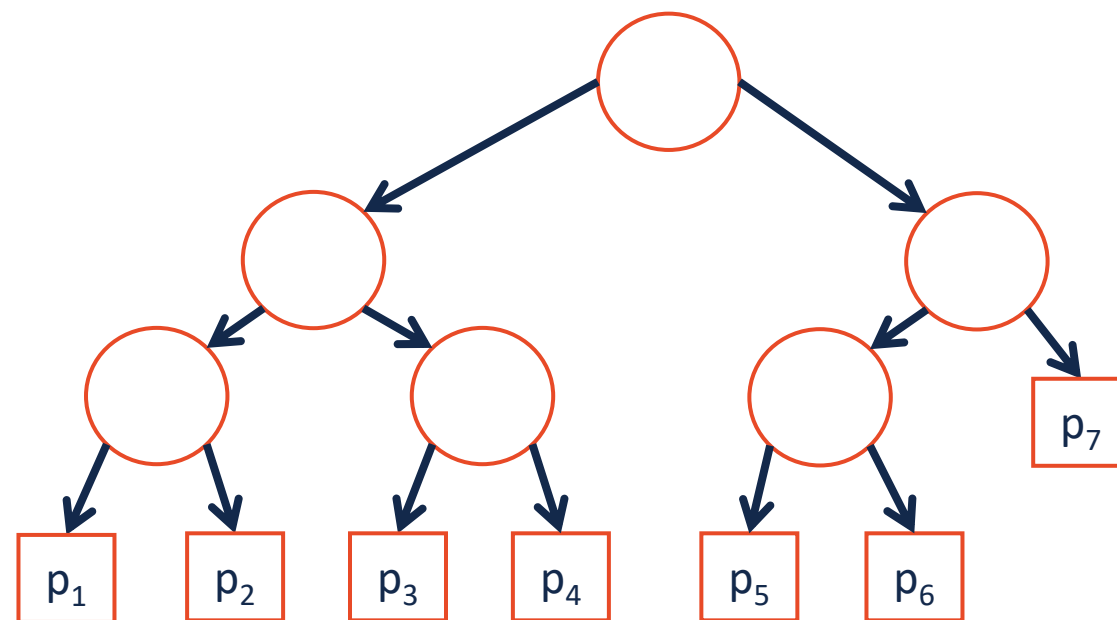
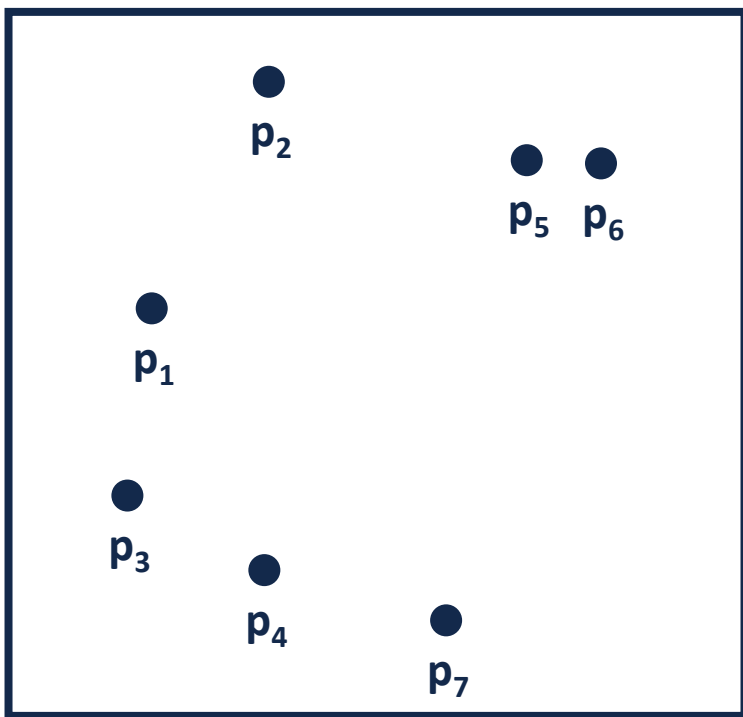
Balanced BST

— Inner nodes are partitions

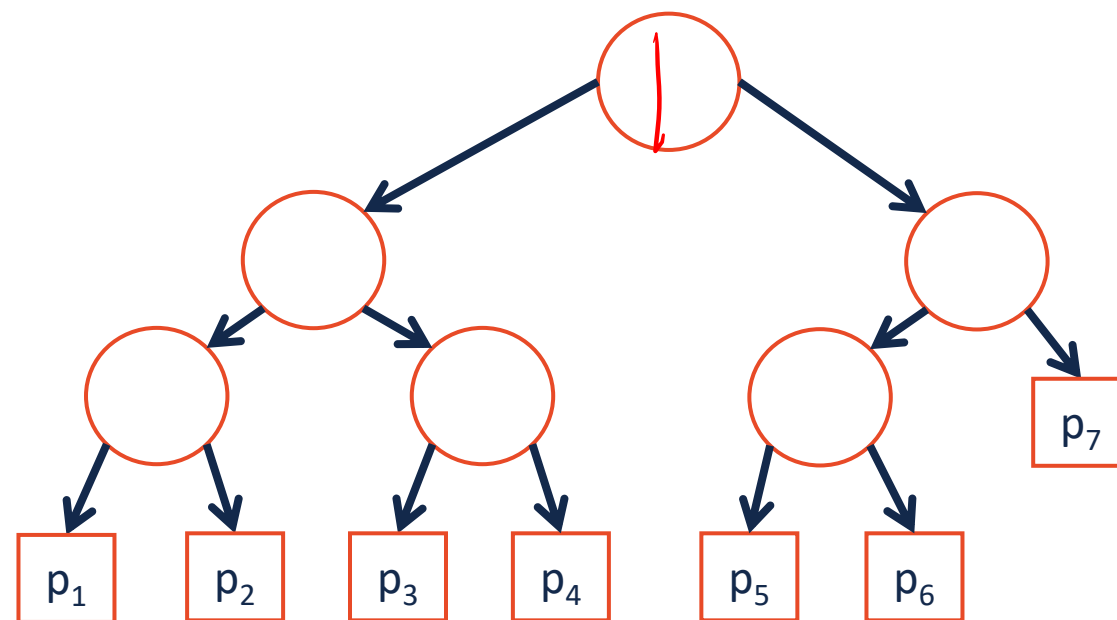
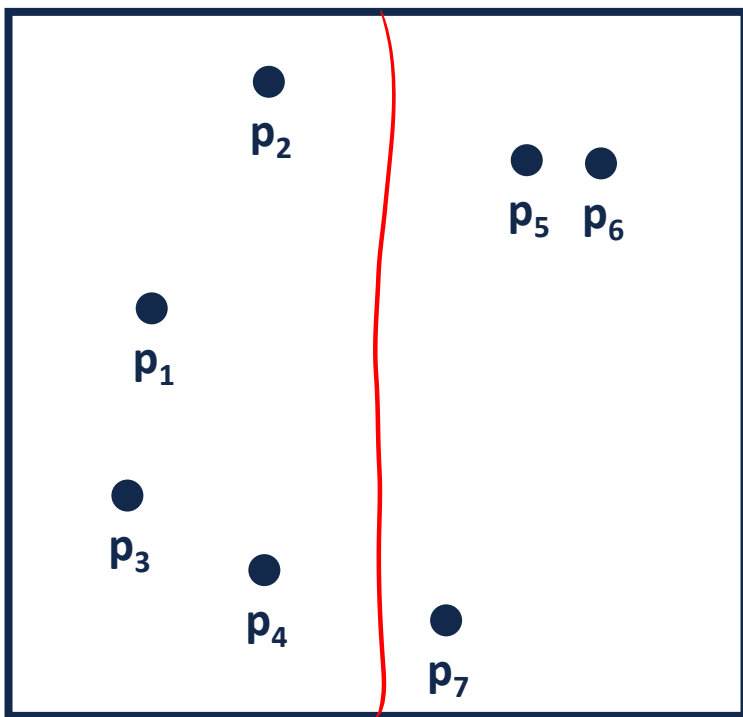
— Leaf nodes are points



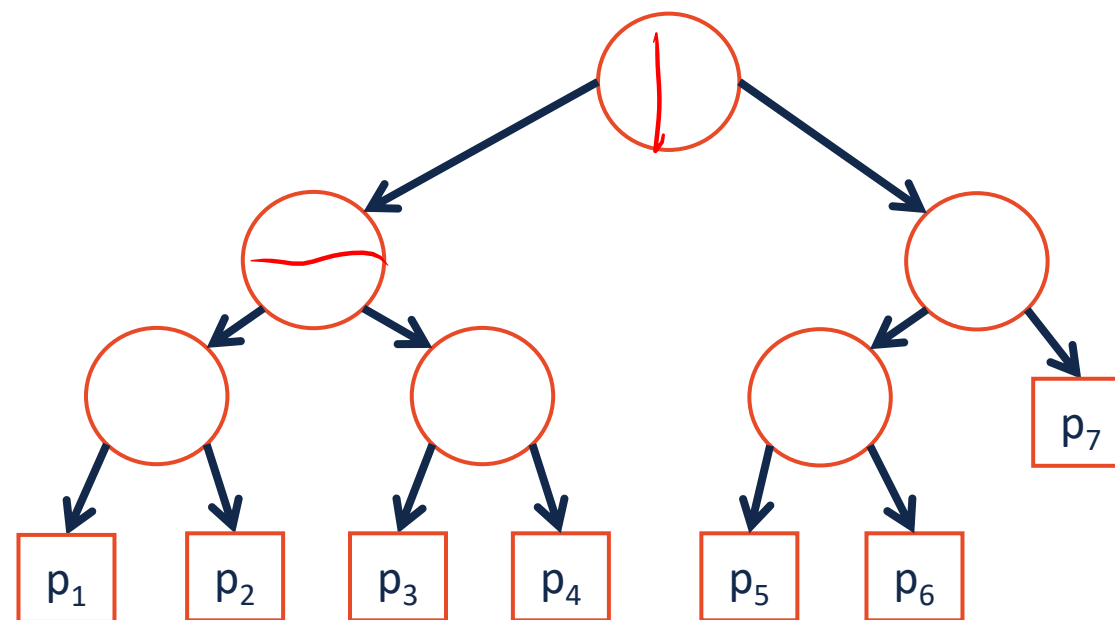
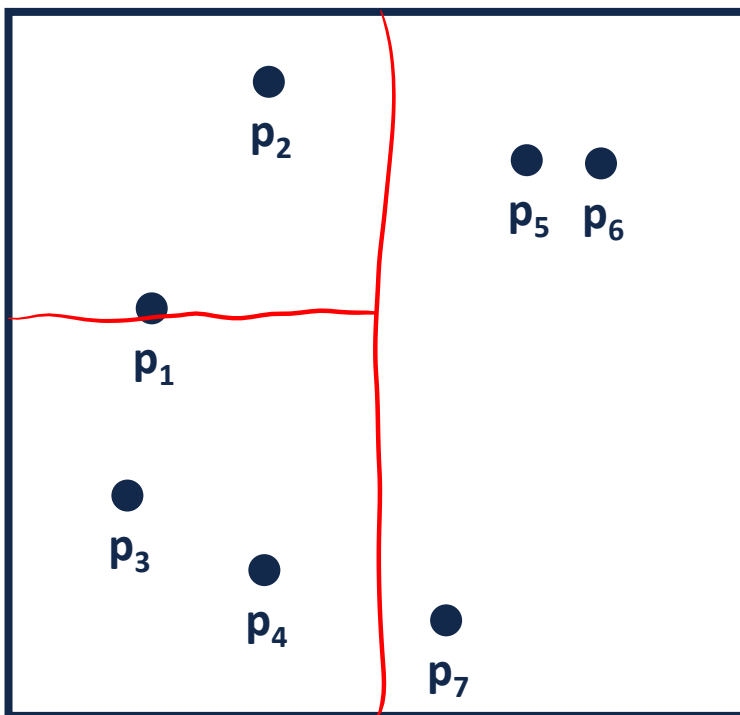
Range-based Searches



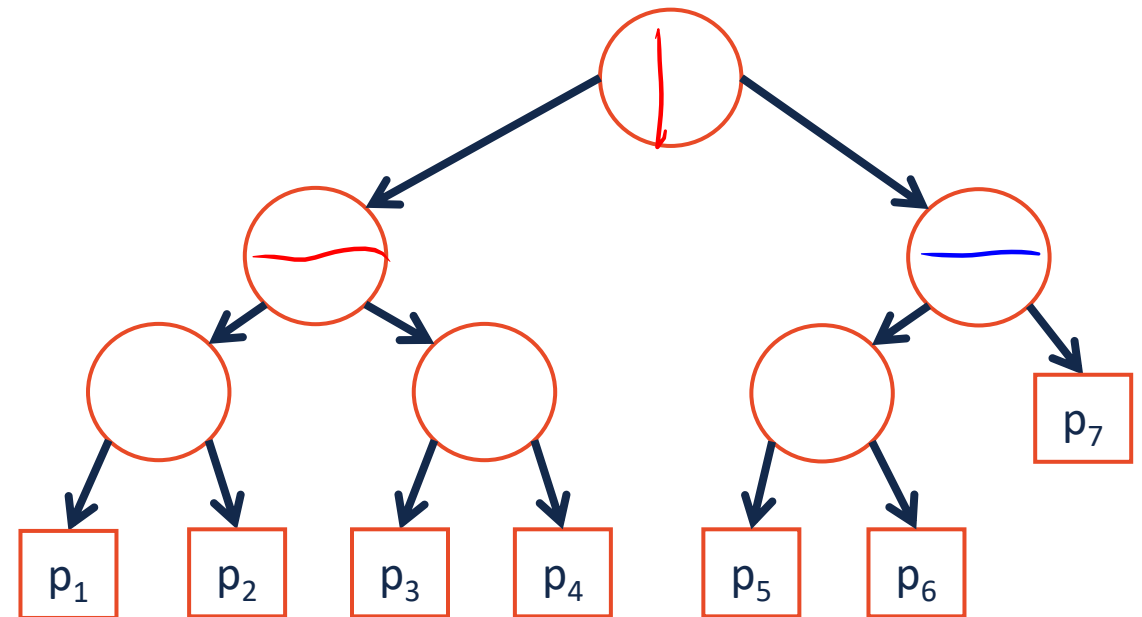
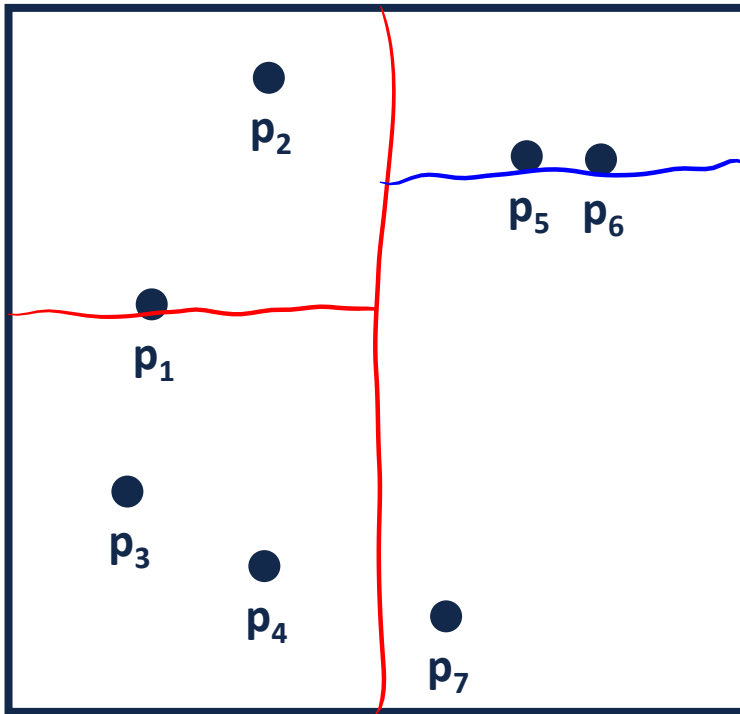
Range-based Searches



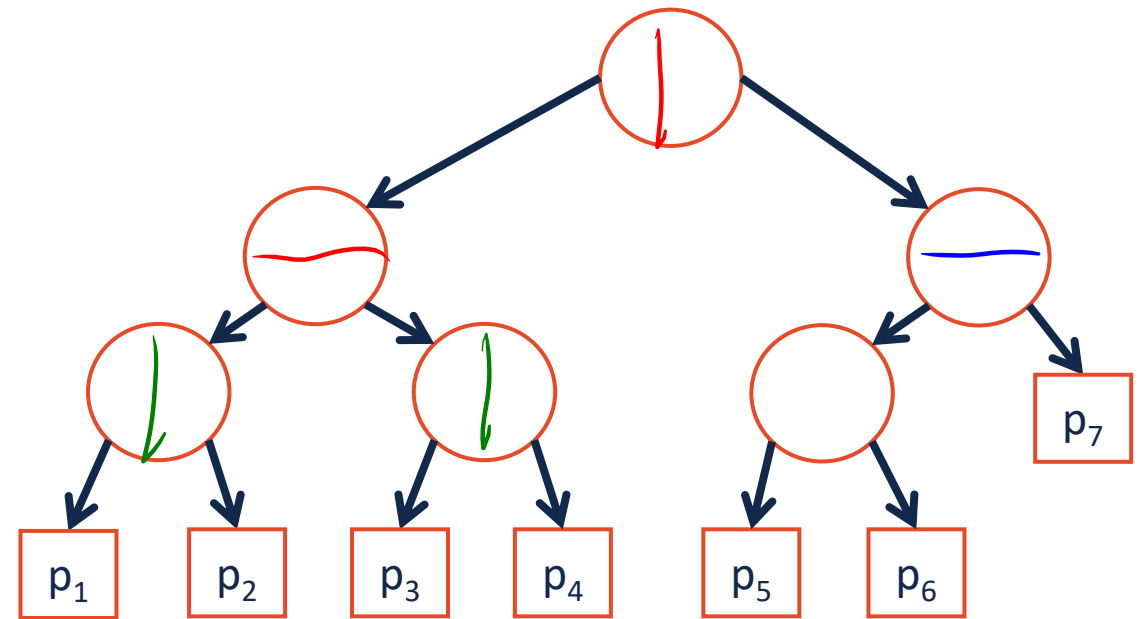
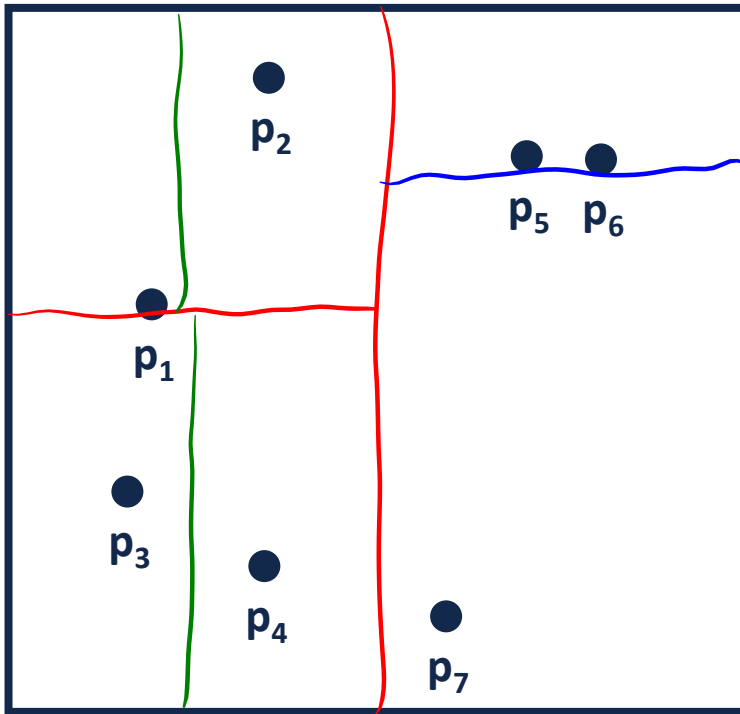
Range-based Searches



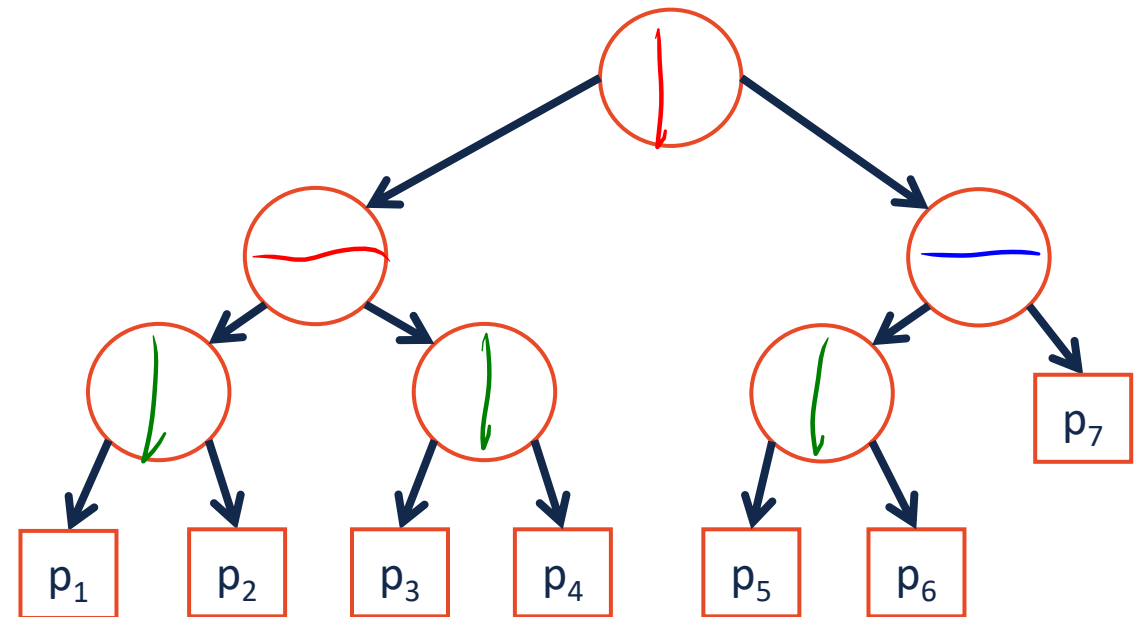
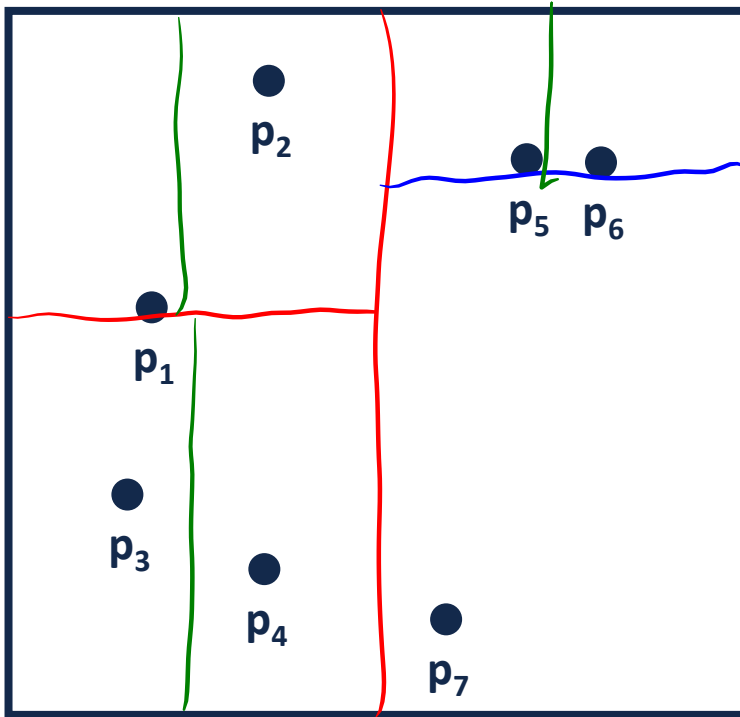
Range-based Searches



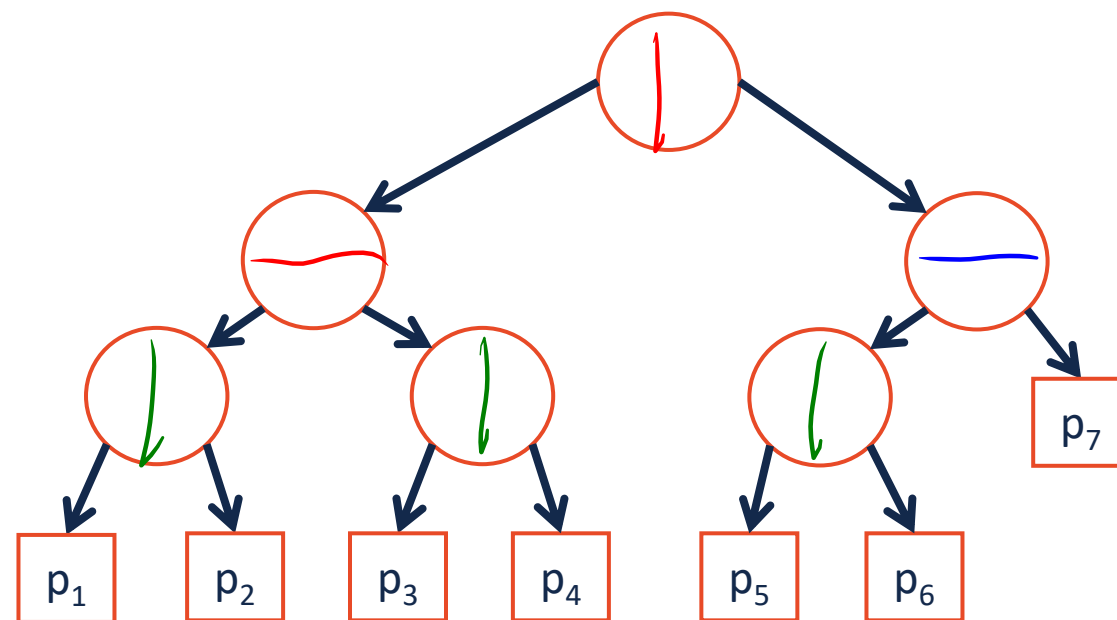
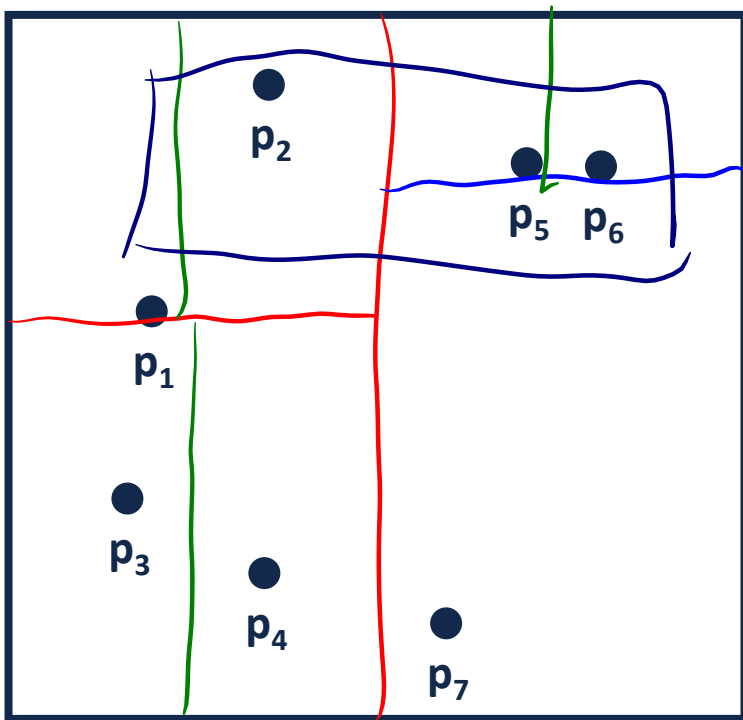
Range-based Searches



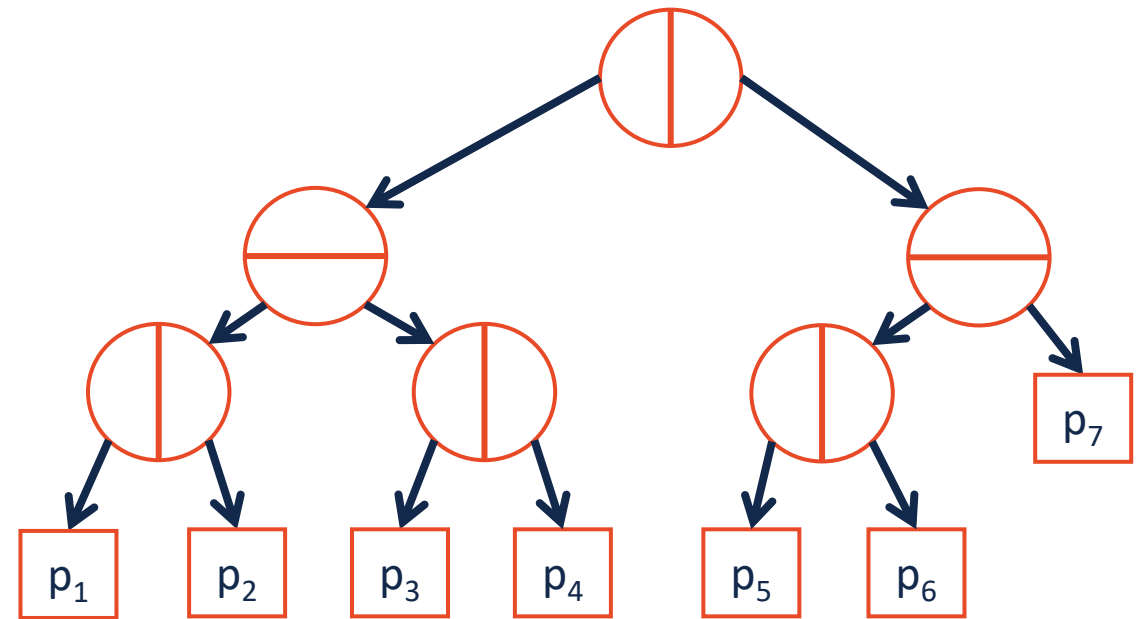
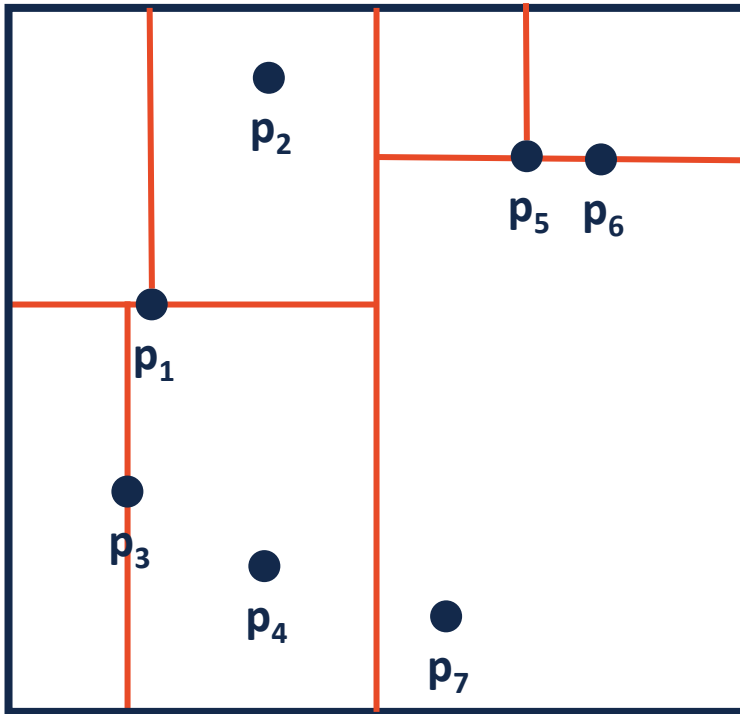
Range-based Searches



Range-based Searches



kD-Trees





CS 225

Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

BTrees

Q: Can we always fit our data in main memory?

No

Q: Where else can we keep our data?

Disk

Cloud

However, big-O assumes uniform time for all operations.

Vast Differences in Time

A **3GHz** CPU performs 3M operations in 1ms

Old Argument: "Disk Storage is Slow"

- Bleeding-edge storage is pretty fast:

NVMe (M.2, PCIe 3.0 x4):

Read 3500 MBps
Write 2500 MBps

- Large Disks (10 TB+) still have slow throughput:

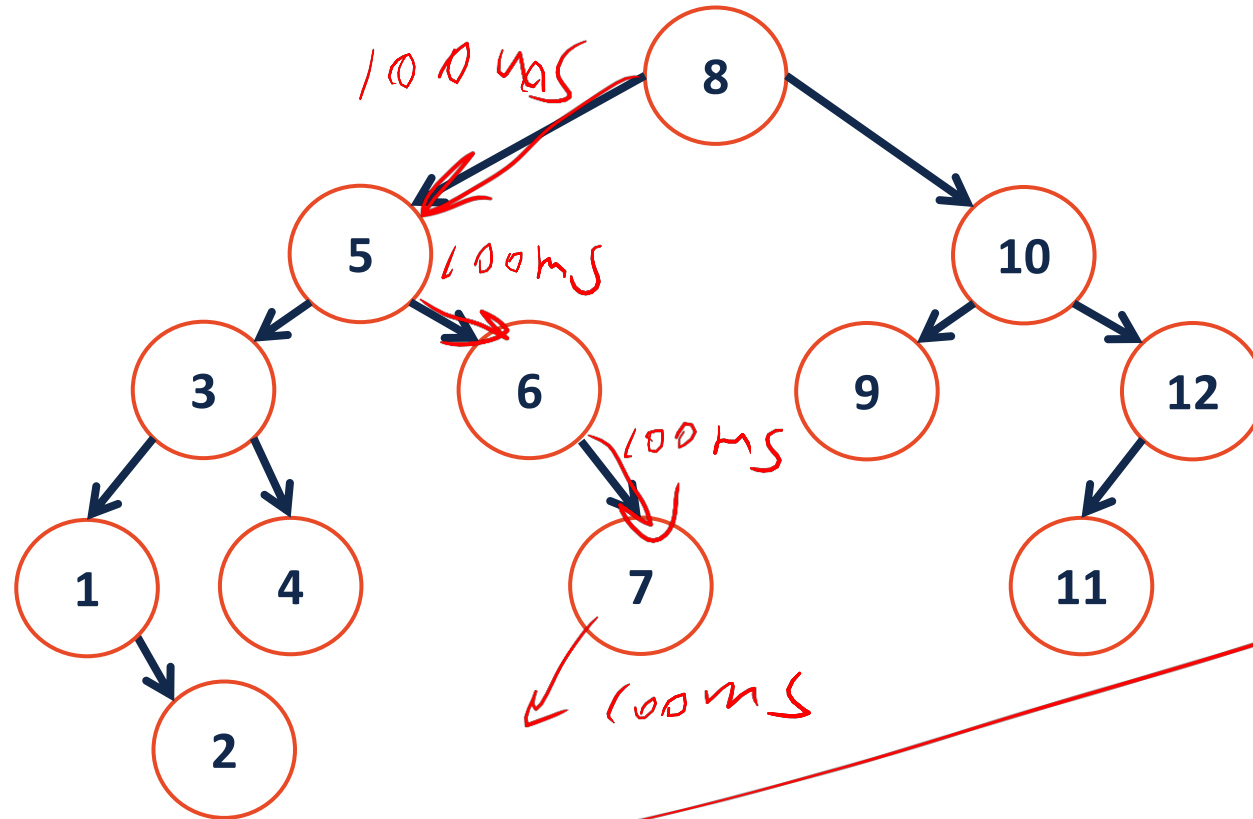
HDD SAS 12GB/s interface 255 MBps

New Argument: "The Cloud is Slow!"

latency 100ms

AVLs on Disk

_ find (6.5)



400ms Gst w/ n=12

Real Application

Imagine storing driving records for everyone in China:

How many records?

500m

How much data in total?

$$5\text{mb} \times 500\text{m} = 2500\text{m MB}$$

How deep is the AVL tree?

$$= 2.5\text{ kTB}$$

$$= 2.5\text{ PB}$$

↓ $h \approx 30$
3 sec

$$\lg_2(500\text{m})$$

BTree Motivations

Knowing that we have large seek times for data, we want to:

1. keep trees short
2. Make sure data is relevant

BTree (of order m)

-3	8	23	25	31	42	43	55
----	---	----	----	----	----	----	----

m=9

Sorted array
BTreeNode

Goal: Minimize the number of reads!

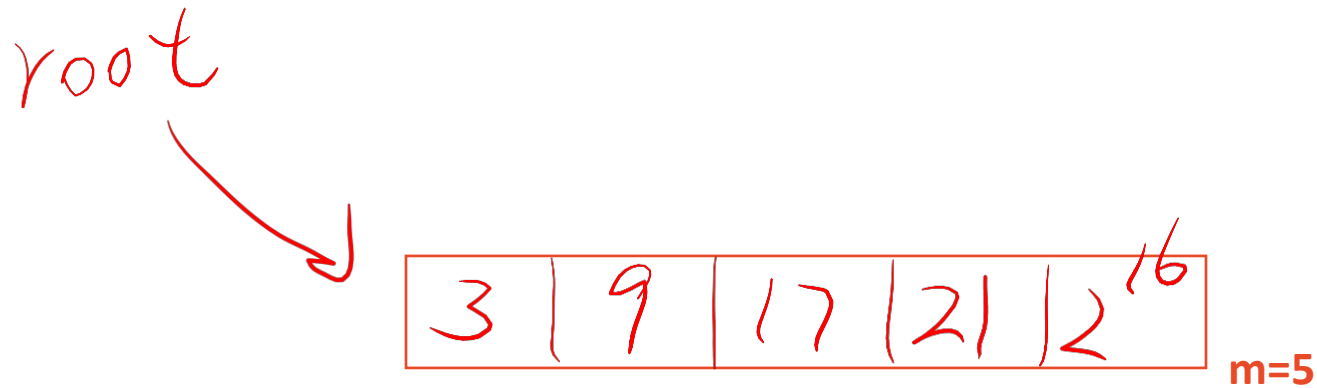
Build a tree that uses

1500 B / node
[1 network packet]
[1 disk block]

BTree Insertion

A **BTree** of order **m** is an m-way tree:

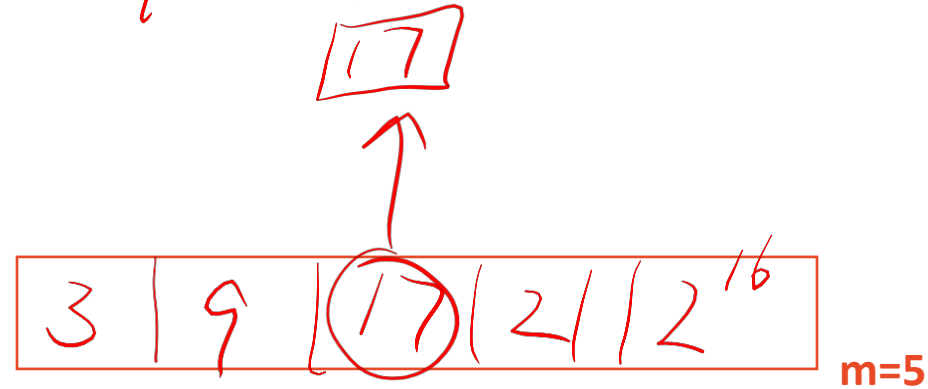
- All keys within a node are ordered
- All leaves hold no more than **m-1** nodes.



BTree Insertion

When a BTree node reaches m keys

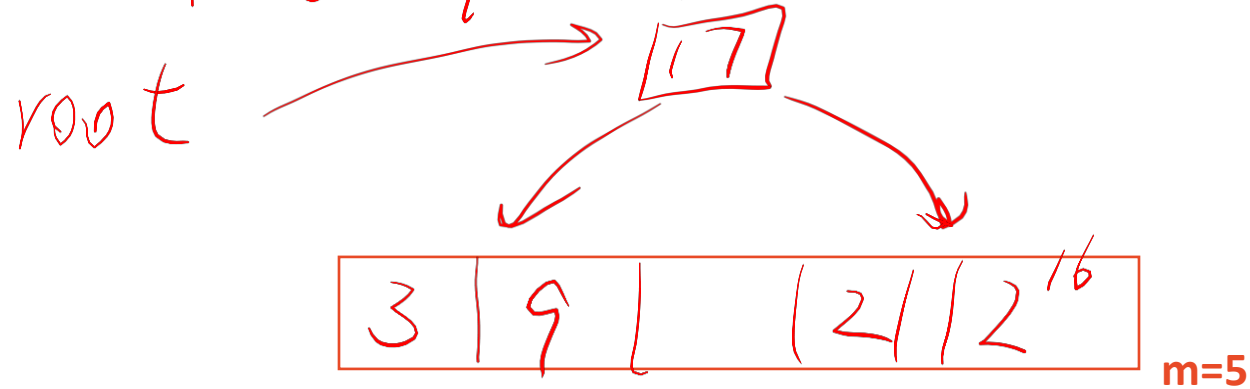
- Split the node
- "Throw up" the middle node



BTree Insertion

When a BTree node reaches m keys

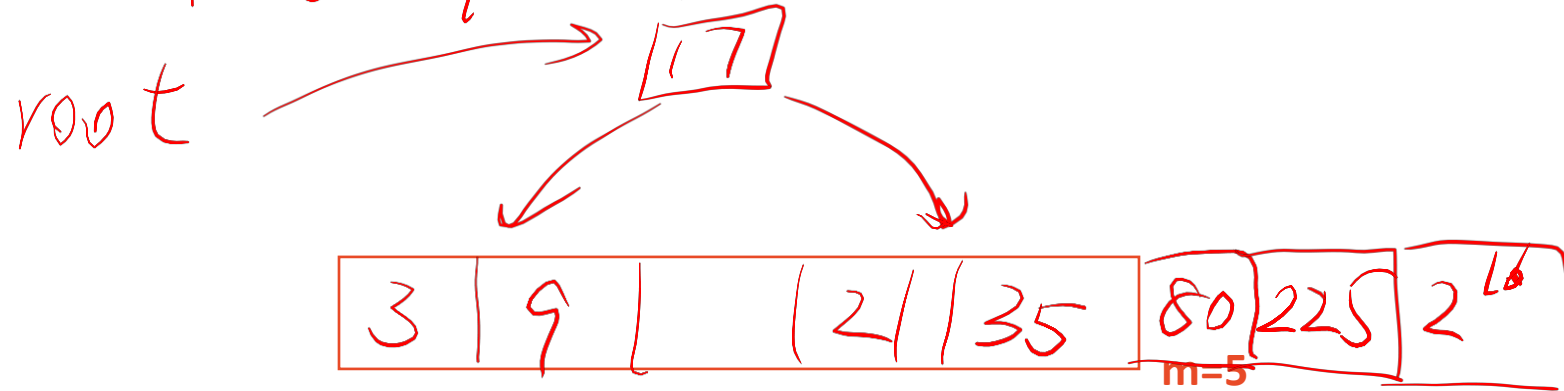
- Split the node
- "Throw up" the middle node



BTree Insertion

When a BTree node reaches m keys

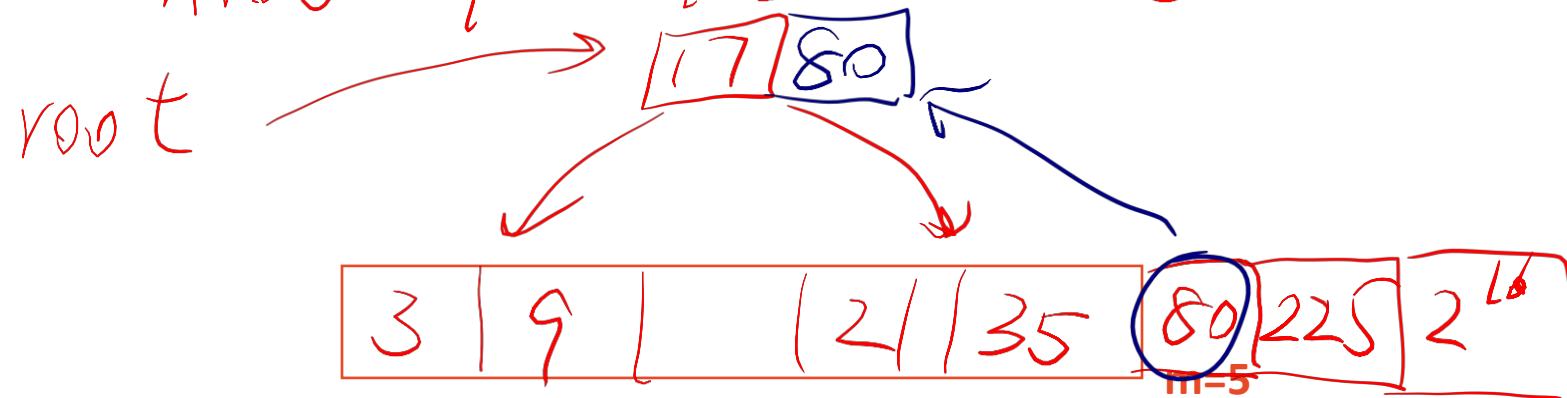
- Split the node
- "Throw up" the middle node



BTree Insertion

When a BTree node reaches m keys

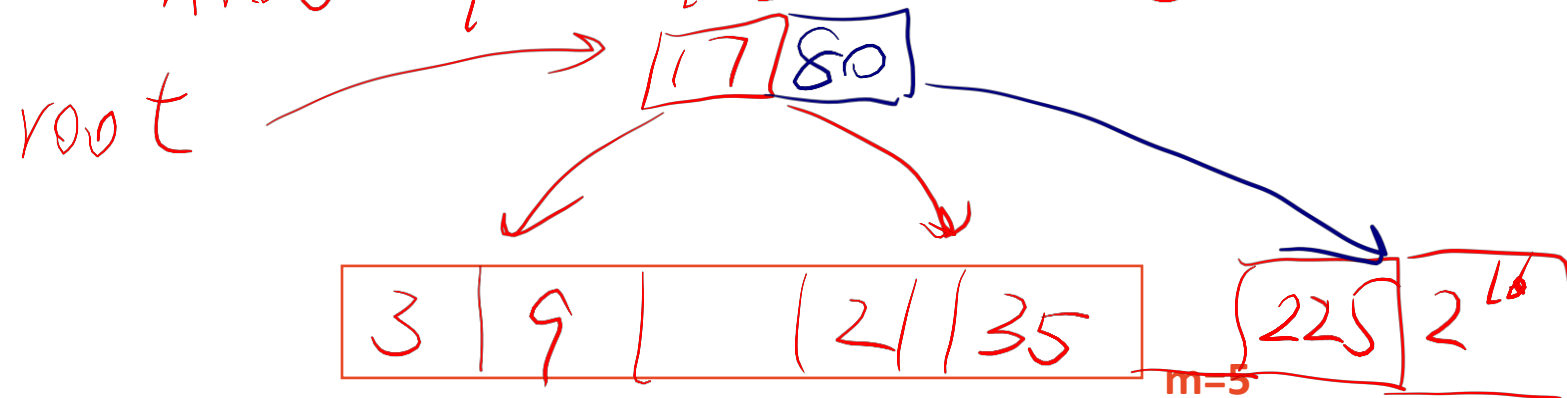
- Split the node
- "Throw up" the middle node



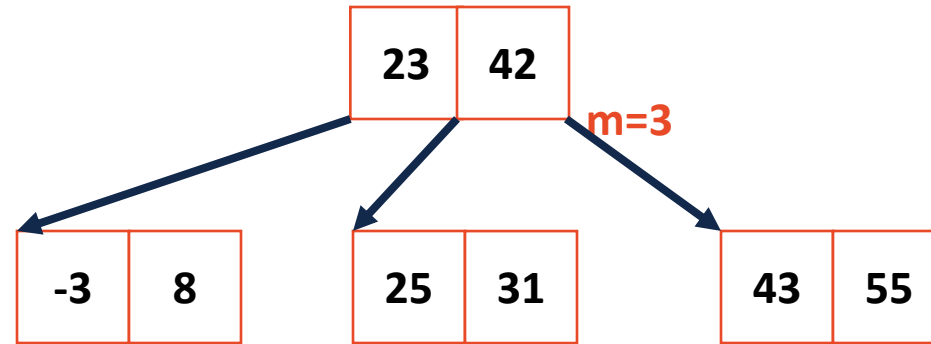
BTree Insertion

When a BTree node reaches m keys

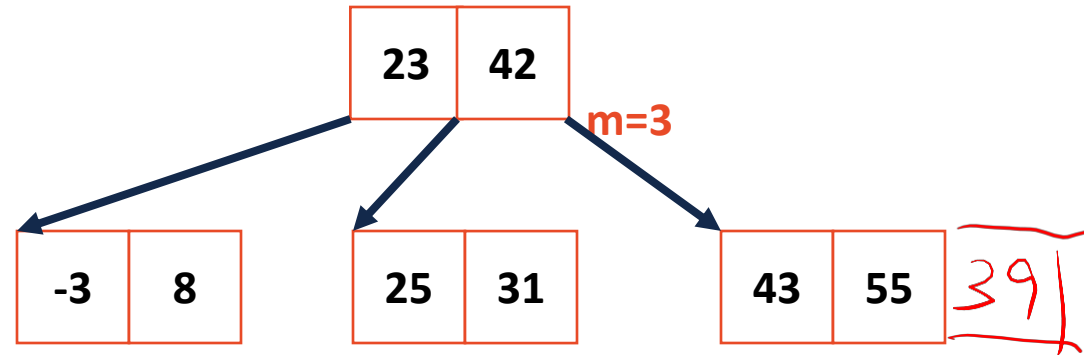
- Split the node
- "Throw up" the middle node



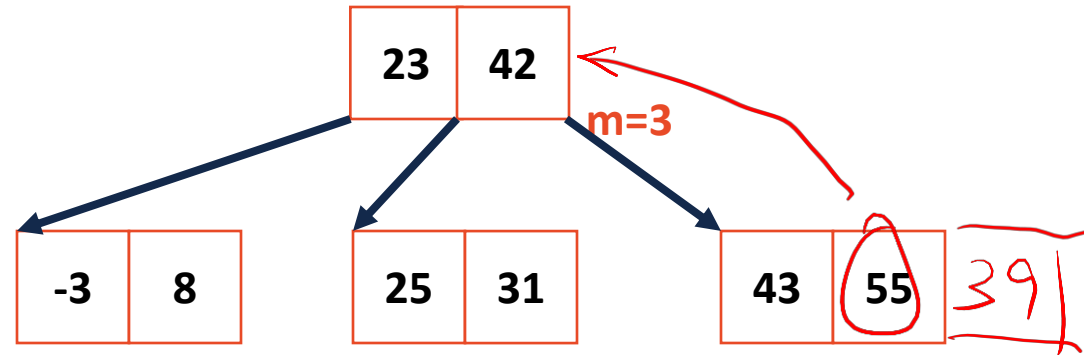
BTree Recursive Insert



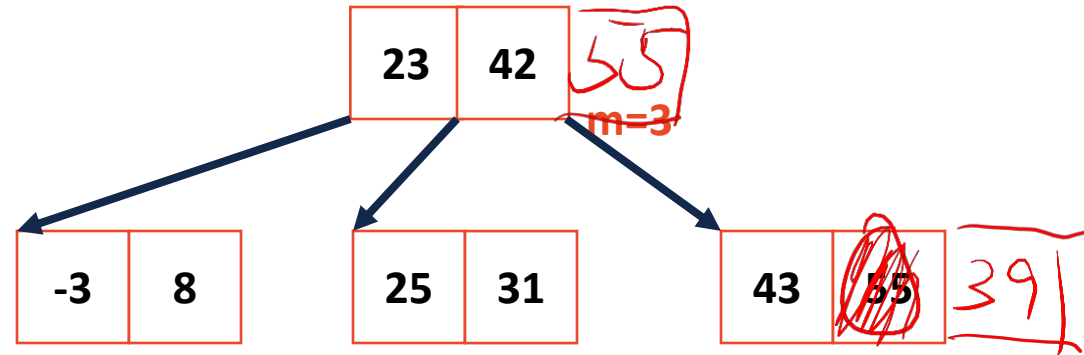
BTree Recursive Insert



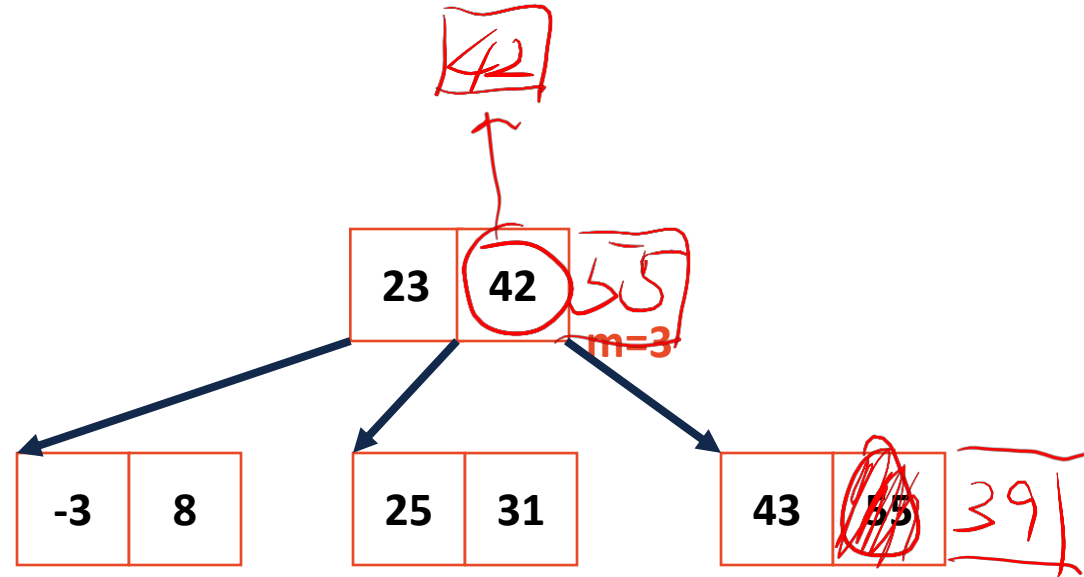
BTree Recursive Insert



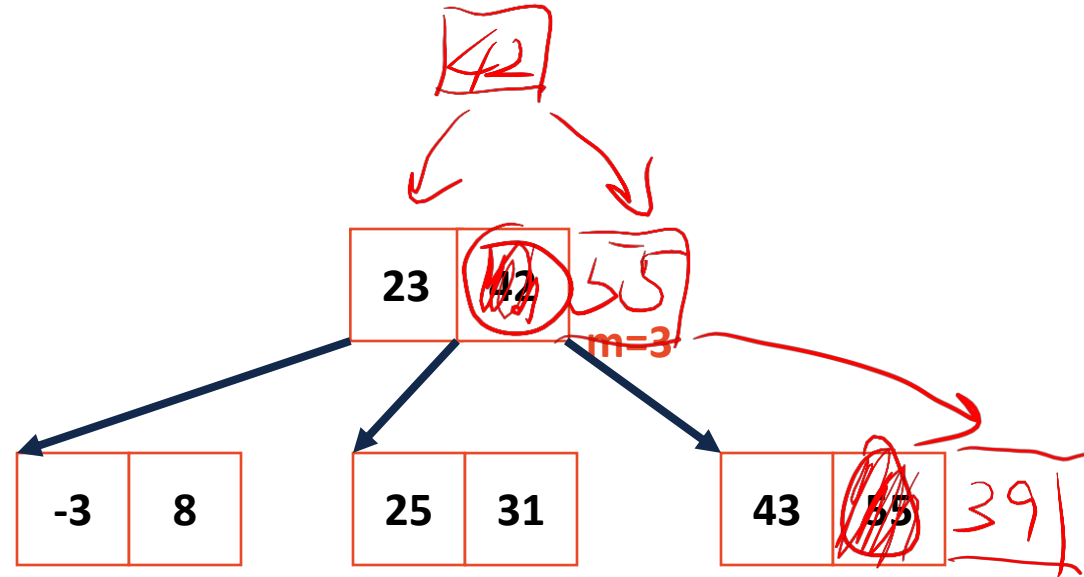
BTree Recursive Insert



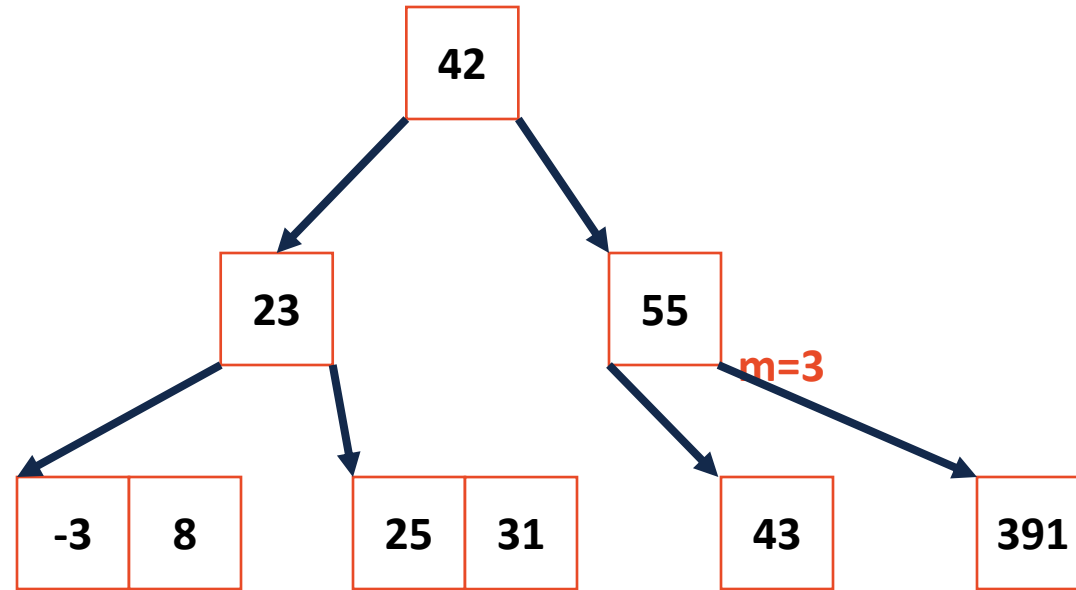
BTree Recursive Insert



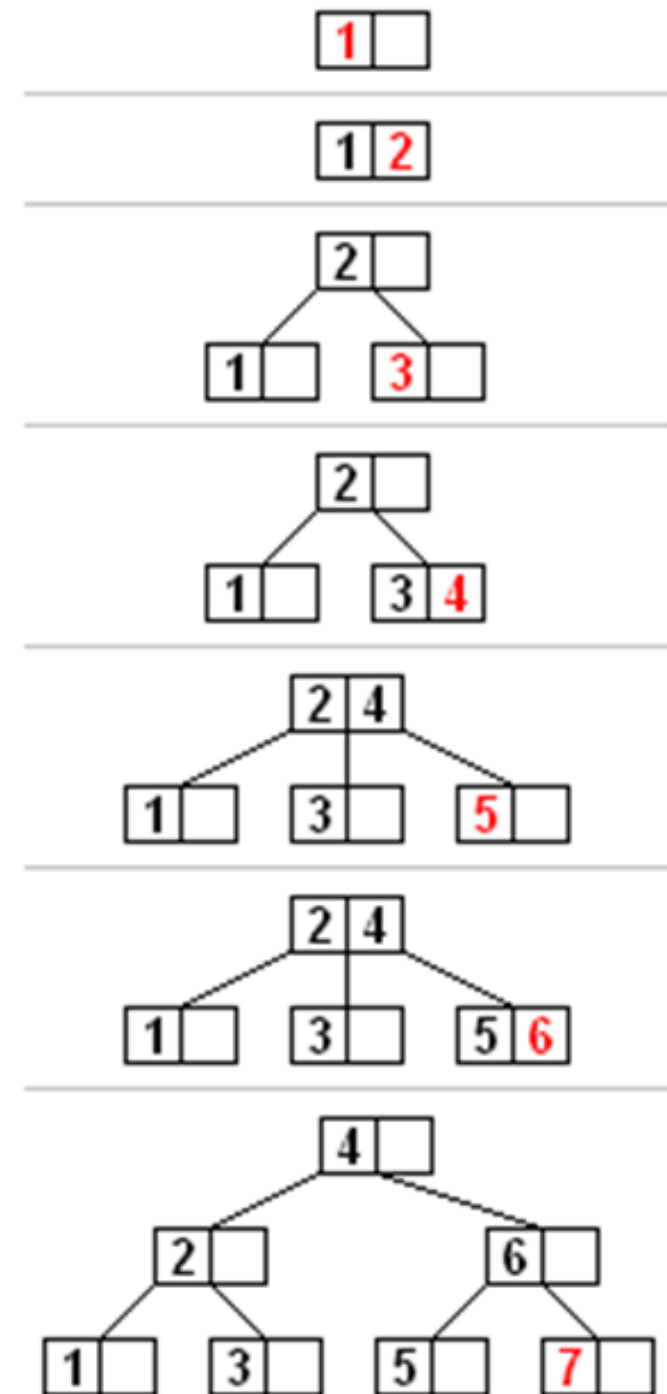
BTree Recursive Insert



BTree Recursive Insert

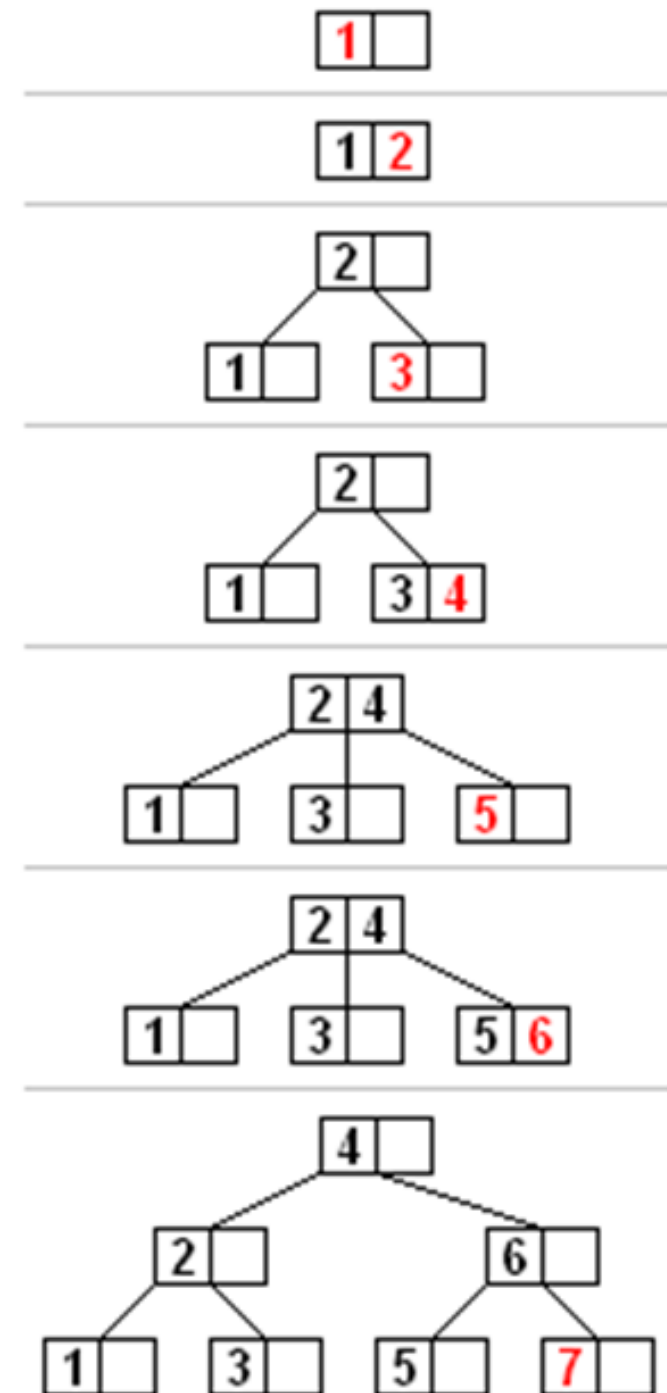


BTree Recursive Insert



BTree Recursive Insert

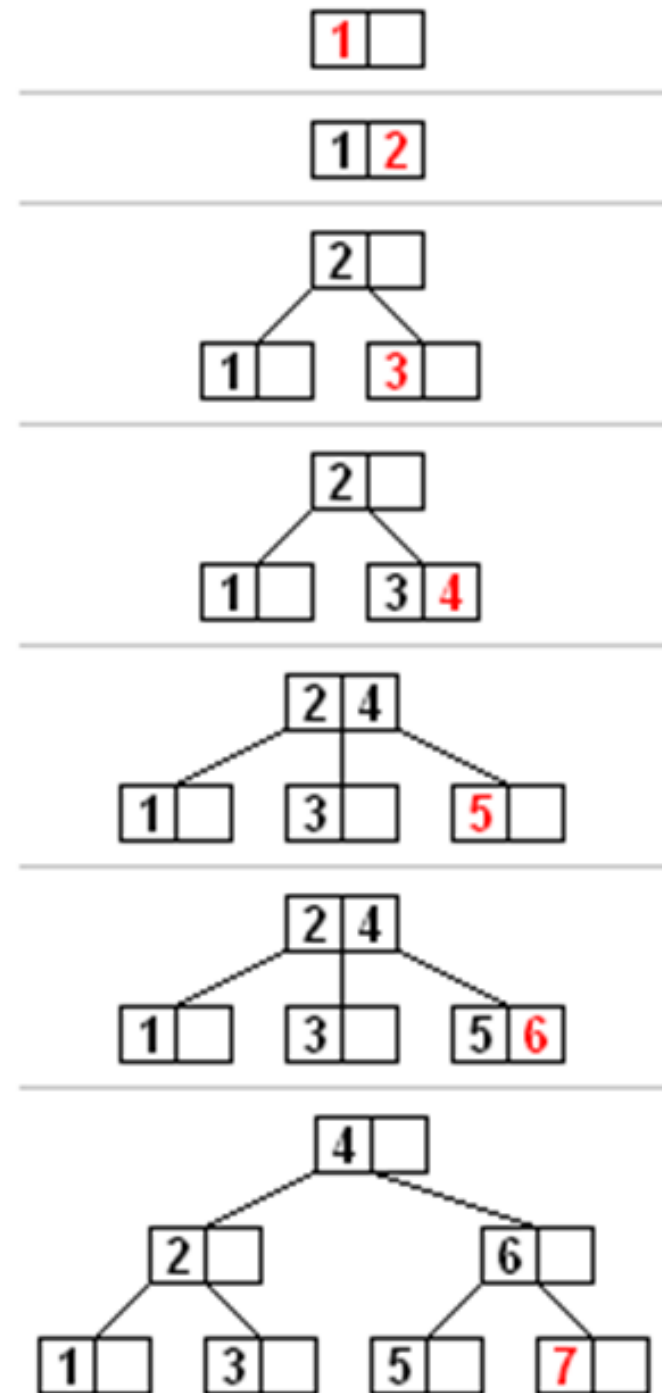
$m = \cancel{2} 3$



BTree Recursive Insert

$$m = \cancel{2} 3$$

All insertions start
at a leaf node



BTree Visualization/Tool

<https://www.cs.usfca.edu/~galles/visualization/BTree.html>

BTree Properties

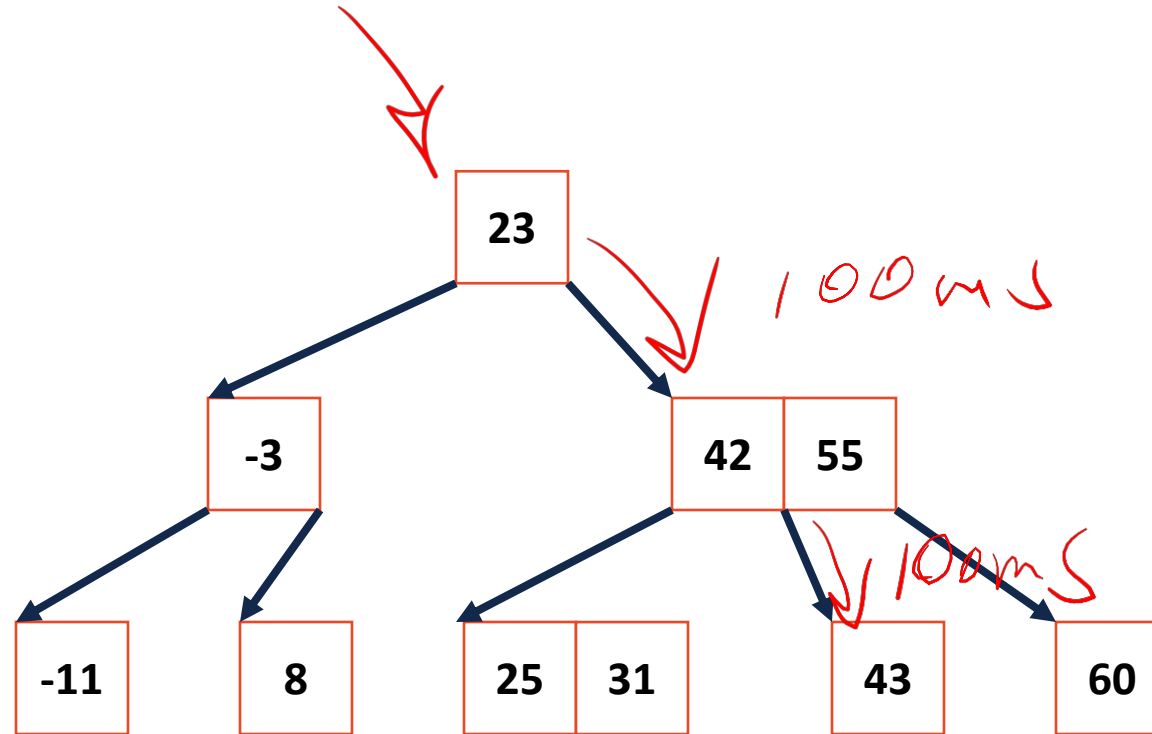
A **BTree** of order **m** is an m-way tree:

- All keys within a node are ordered
- All leaves hold no more than **m-1** nodes.
- All internal nodes have exactly **one more child than key**.
- Root nodes can be a leaf or have **[2, m]** children. $\rightarrow [1, m-1]$ keys
- [1, m-1] keys
- All non-root, internal nodes have **[ceil(m/2), m]** children. $\downarrow$
 $\lceil m/2 \rceil$
 $\downarrow$
 $[\lceil m/2 \rceil - 1, m-1]$ keys
- [ceil(m/2)-1, m-1] keys
- All leaves are on the same level

BTree Search

- find (43)

$m=3$



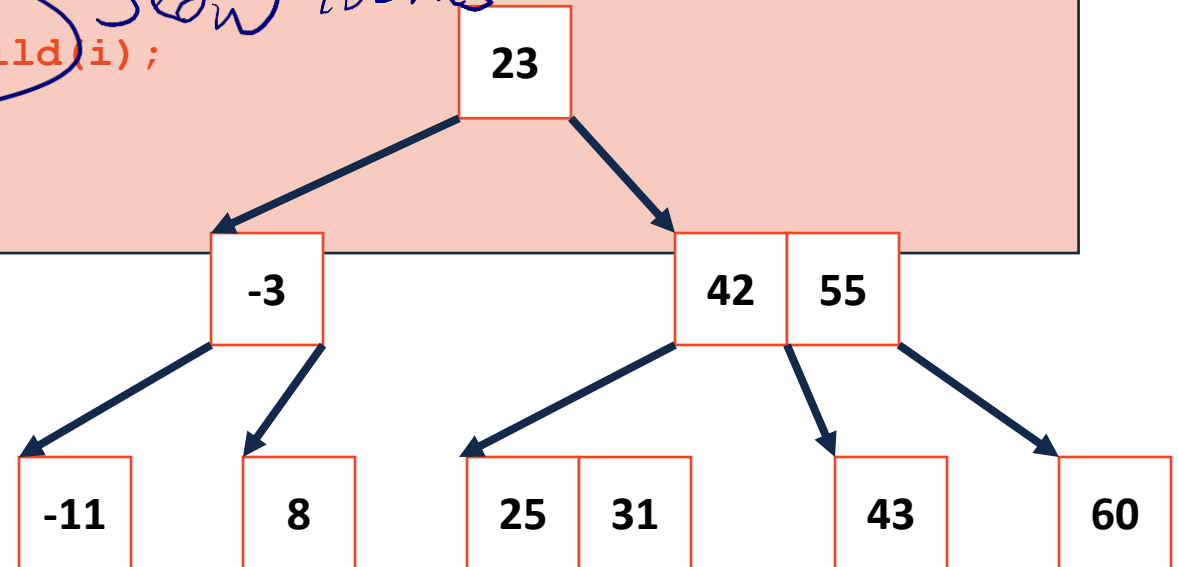
BTree Search

```
1 bool Btree::_exists(BTreeNode & node, const K & key) {  
2     // Goes through a node, find location  
3     unsigned i;  
4     for ( i = 0; i < node.keys_ct_ && key < node.keys_[i]; i++) {}  
5  
6     if ( i < node.keys_ct_ && key == node.keys_[i] ) {  
7         return true;  
8     }  
9  
10    if ( node.isLeaf() ) { } Base case  
11        return false;  
12    } else {  
13        BTreeNode nextChild = node._fetchChild(i);  
14        return _exists(nextChild, key);  
15    }  
16 }
```

Can do BSearch

Slow loops

Recursion



BTree Analysis

The height of the BTree determines maximum number of
fetches possible in data search.

Disk reads, done

...and the height of the structure is: $\log_m n$.

$$O(\log_m n) = O(\lg n)$$

Therefore: The number of seeks is no more than $\log_m n$.

...suppose we want to prove this!

CS 225

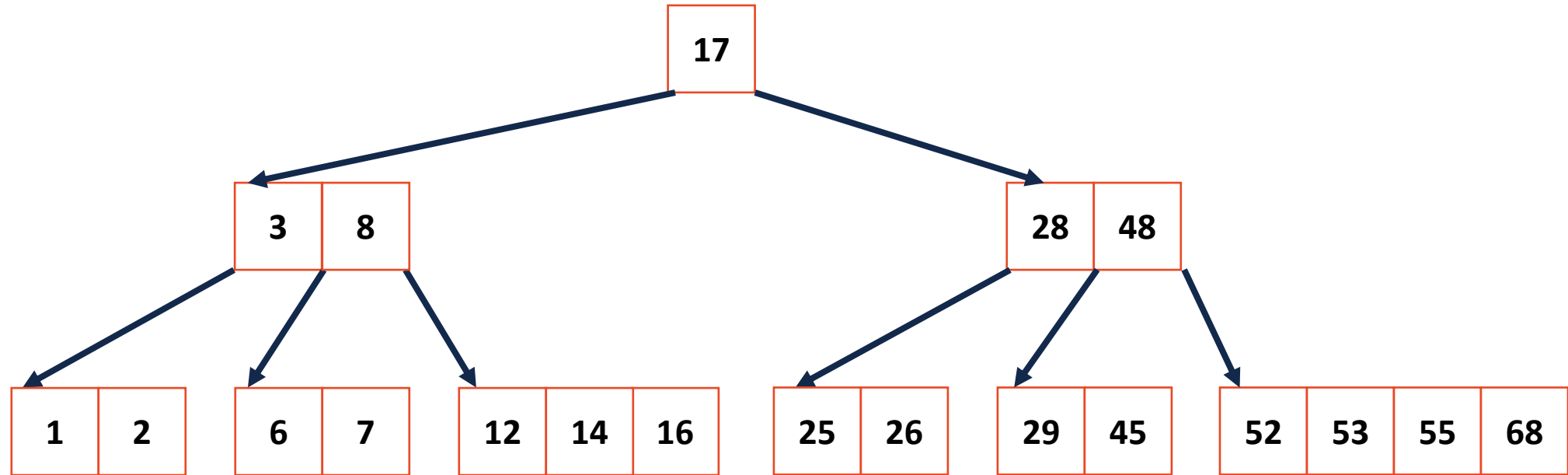
Data Structures

Volodymyr Kindratenko

Slides courtesy of Wade Fagen-Ulmschneider

BTree

order?



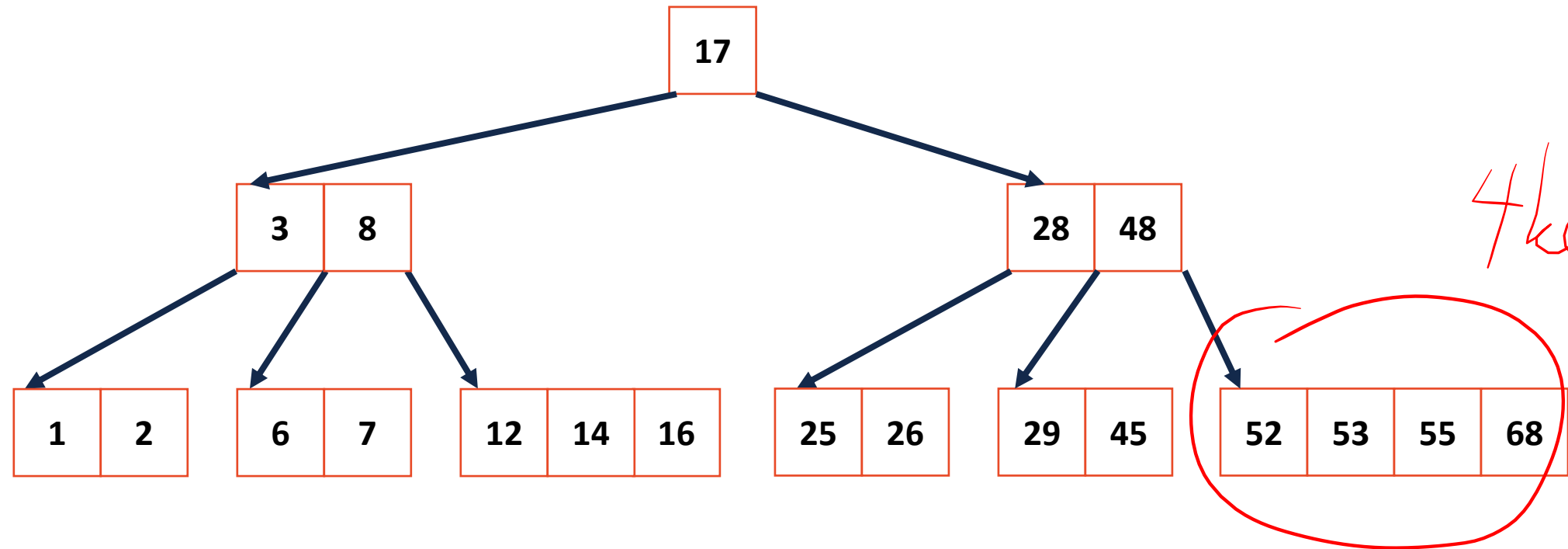
Btree Properties

A **BTree** of order **m** is an m-way tree:

- All keys within a node are ordered
- All nodes contain no more than **m-1** keys.
- All internal nodes have exactly **one more child than keys**
- Root nodes can be a leaf or have **[2, m]** children.
- All non-root, internal nodes have **[ceil(m/2), m]** children.
- All leaves are on the same level

BTree

order ? > 4



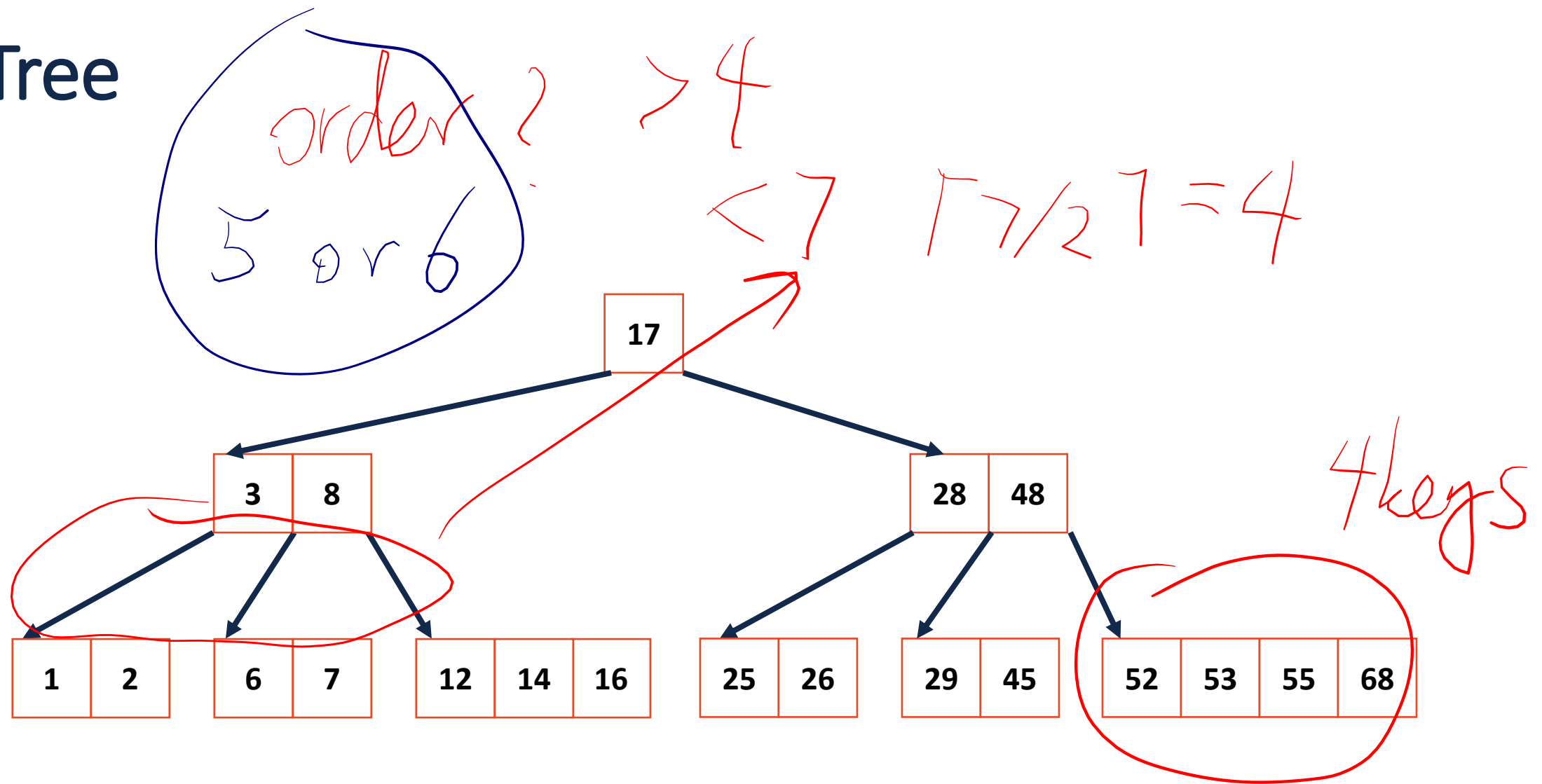
4 keys

Btree Properties

A **BTree** of order **m** is an m-way tree:

- All keys within a node are ordered
- All nodes contain no more than **m-1** keys.
- All internal nodes have exactly **one more child than keys**
- Root nodes can be a leaf or have **[2, m]** children.
- All non-root, internal nodes have **[ceil(m/2), m]** children.
- All leaves are on the same level

BTree



Btree Properties

A **BTree**s of order **m** is an m-way tree:

- All keys within a node are ordered
- All nodes contain no more than **m-1** keys.
- All internal nodes have exactly **one more child than keys**
- Root nodes can be a leaf or have **[2, m]** children.
- All non-root, internal nodes have **[ceil(m/2), m]** children.
- All leaves are on the same level

BTree Analysis

The height of the BTree determines maximum number of
fetch/seek possible in search data.

...and the height of the structure is: $O(\log_m(n))$.

$O(\log n)$?
Therefore: The number of seeks is no more than $\log_m n$.

...suppose we want to prove this!

BTree Analysis

In our AVL Analysis, we saw finding an upper bound on the height (given n) is the same as finding a lower bound on the nodes (given h).

$$h \leq f(n)$$

We want to find a relationship for BTrees between the number of keys (n) and the height (h).

$$f(h) \leq n$$

BTree Analysis

Strategy:

We will first count the number of nodes, level by level.

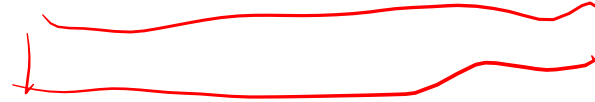
Then, we will add the minimum number of keys per node (n).

The minimum number of nodes will tell us the largest possible height (h), allowing us to find an upper-bound on height.

BTree Analysis

The minimum number of **nodes** for a BTree of order m **at each level:**

root:



level 1:

level 2:

level 3:

...

level h :

Btree Properties

A **BTree** of order **m** is an m-way tree:

- All keys within a node are ordered
- All nodes contain no more than **m-1** keys.
- All internal nodes have exactly **one more child than keys**
- Root nodes can be a leaf or have **[2, m]** children.
- All non-root, internal nodes have **[ceil(m/2), m]** children.
- All leaves are on the same level

BTree Analysis

$$\text{Let } t = \lceil m/2 \rceil$$

The minimum number of **nodes** for a BTree of order m at each level:

root:

level 1:

level 2:

level 3:

...

level h :

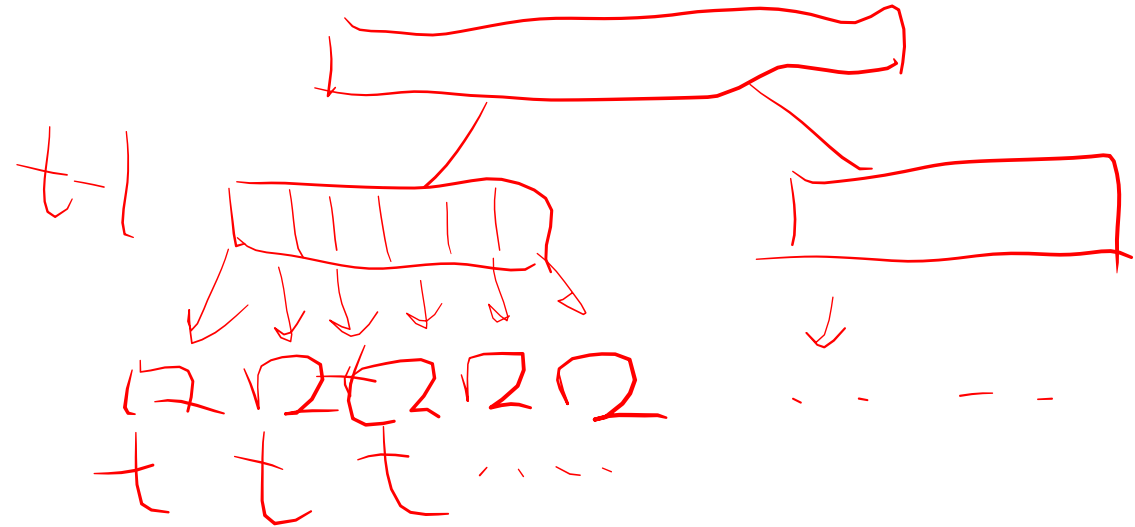
1

2

$2t$

$2t^2$

$2t^{h-1}$



BTree Analysis

The **total number of nodes** is the sum of all of the levels:

BTree Analysis

The total number of keys:

Each non-root node has $t-1$ keys

$$1 + 2(t^h - 1) \cdot (\cancel{t-1})$$

$= 2t^h - 1$ min keys
a B tree of order m
given h

BTree Analysis

order of Btree

Given **m=101**, a tree of height **h=4** has:

Minimum Keys:

$$2t^h - 1 \xrightarrow{h=4} 2t^4 - 1 \xrightarrow{t=\lceil m/2 \rceil = 51}$$

$$2(51)^4 - 1 \approx 12.5m$$

Maximum Keys:

$$\boxed{\text{only } 300m}$$

BTree Analysis

The **smallest total number of keys** is:

$$2t^h - 1$$

So an inequality about **n**, the total number of keys:

$$n \geq 2t^h - 1$$

Solving for **h**, since **h** is the number of seek operations:

$$\log_t \left(\frac{n+1}{2} \right) \geq h \rightarrow \log_{(m/2)} \left(\frac{n+1}{2} \right) \geq h$$

$$\log_{(5)} (500k) = 3.37$$

$$m = 101$$
$$h = \lceil m \rceil$$

BTree Analysis

order of Btree

Given **m=101**, a tree of height **h=4** has:

Minimum Keys:

$$2t^h - 1 \xrightarrow{h=4} 2t^4 - 1 \xrightarrow{t=\lceil m/2 \rceil = 51}$$

$$2(51)^4 - 1 \approx 12.5m$$

Maximum Keys:

$$\boxed{\text{only } 300m}$$

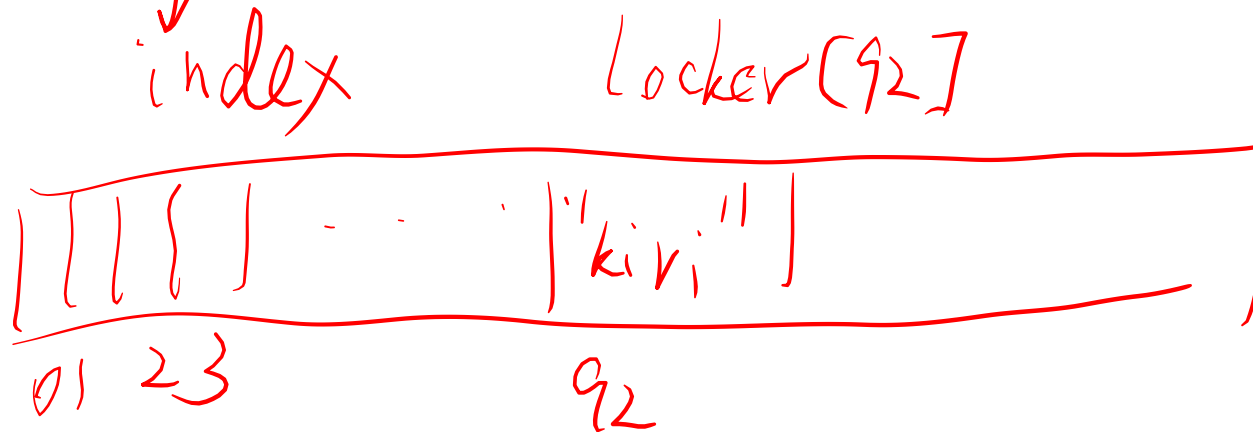
Prove yourself!

Hashing

Locker Number	Name
103	Rick
92	Kiri
330	Mary Catherine
46	Blake
124	Erin

$O(1)$ ✓

$O(X)$
keyspace



Hashing

Commonly:

Just use a “hash table” to implement a dictionary.

Dictionary ADT

Data is often organized into key/value pairs:

UIN → Advising Record

Course Number → Lecture/Lab Schedule

Node → Incident Edges

Flight Number → Arrival Information

URL → HTML Page

...

Dictionary.h

```
1 #ifndef DICTIONARY_H
2 #define DICTIONARY_H
3
4 template <class K, class V>
5 class Dictionary {
6     public:
7         void insert(K & k, V & v);
8         void remove(const K & k);
9         V & find(const K & k) const;
10
11
12
13
14
15
16     private:
17 };
18
19 #endif
20
21
22
```

associative array []
V & operator[] (const K & k) const;
d["wade"]

Hashing

Goals:

We want to define a **keyspace**, a (mathematical) description of the keys for a set of data.

...use a function to map the **keyspace** into a small set of integers.

$$h: K \rightarrow N$$